

OOP PART-01

Object Oriented Programming

Object-oriented programming (OOP) is a computer programming model that organizes software design around data, or objects, rather than functions and logic. An object can be defined as a data field that has unique attributes and behaviour.

Classes

Classes are user-defined data types that act as the blueprint for individual objects, attributes and methods.

Discuss the basic structure of a class with an example:

Here's the basic structure of a class:

- **Class Name:** A capitalized name that identifies the class.
- **Attributes:** These store the data (properties) of the object, like color, size, etc.
- **Methods:** These define the actions (behaviors) the object can perform, like greet, calculate area, etc.

EX:

```
public class Car {  
  
    // Attributes (member variables)  
  
    String color;  
  
    int wheels;  
  
    // Method (member function)  
  
    public void drive() {  
  
        System.out.println("The car is driving!");  
  
    }  
  
}
```

Describe the benefits of using constructor in a class by using an example:

In Java, a constructor is a block of codes similar to the method. It is a special type of method which is used to initialize the object.

Benefits of Constructors:

1. **Initialization:** Sets up objects when they are created, making sure everything is ready to use.
2. **Encapsulation:** Keeps data safe by securely setting up private fields.
3. **Convenience:** Makes creating objects easier by avoiding repetitive setup steps.
4. **Readability:** Clearly shows how to create objects and what their starting state is.

EX:

```
public class Car {  
    // Attributes (fields) of the Car class  
    private String make;  
    private String model;  
    private int year;  
  
    // Constructor to initialize the attributes  
    public Car(String make, String model, int year) {  
        this.make = make;  
        this.model = model;  
        this.year = year;  
    }  
    // Main method to demonstrate creating and using a Car object  
    public static void main(String[] args) {  
        // Creating a Car object with attributes  
        Car car = new Car("Toyota", "Camry", 2020);  
        // Using the object to call its method and print its attributes  
        System.out.println("Car: " + car.make + " " + car.model + " " + car.year);  
    }  
}
```

Explanation:

- **Class Definition:** Defines a `Car` class with three private fields: `make`, `model`, and `year`.

- **Constructor:** Initializes these fields in a single line constructor.
- **Main Method:** Demonstrates creating a `Car` object (`car`) using the constructor and then prints its `make`, `model`, and `year`.

Objects

Objects are instances of a class created with specifically defined data. When class is defined initially, the description is the only object that is defined.

EX: Follow class 2nd ex.

Methods

Methods are functions that are defined inside a class that describe the behaviours of an object. Each method contained in class definitions starts with a reference to an instance object.

EX:

```
public class Car {

    // Properties (member variables)

    String color;

    int wheels;

    // Method (member function)

    public void drive() {

        System.out.println("The car is driving!");

    }

}
```

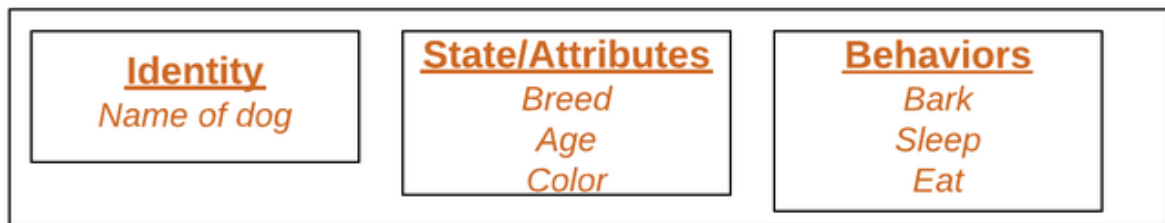
Attributes

Attributes are defined in the class template and represent the state of an object.

Objects will have data stored in the attributes field. Class attributes belong to the class itself

EX: Follow class 2nd ex.

Example: “Dog” is a real-life Object, which has some characteristics like color, Breed, Bark, Sleep, and Eats.



OOPs Concepts:

- Class
- Objects
- Data Abstraction
- Encapsulation
- Inheritance
- Polymorphism
- Dynamic Binding
- Message Passing

What are the benefits of OOP?

Benefits of OOP include:

- **Modularity.** Encapsulation enables objects to be self-contained, making collaborative development easier.
- **Reusability.** Code can be reused through inheritance, meaning a team does not have to write the same code multiple times.
- **Productivity.** Programmers can construct new programs quicker through the use of multiple libraries and reusable code.
- **Easily upgradable and scalable.** Programmers can implement system functionalities independently.
- **Interface descriptions.** Descriptions of external systems are simple, due to message passing techniques that are used for objects communication.
- **Security.** Using encapsulation and abstraction, complex code is hidden, software maintenance is easier and internet protocols are protected.

- **Flexibility.** Polymorphism enables a single function to adapt to the class it is placed in. Different objects can also pass through the same interface.

Data Abstraction

Data Abstraction is the property by virtue of which only the essential details are displayed to the user. The trivial or the non-essential units are not displayed to the user. Data Abstraction may also be defined as the process of identifying only the required characteristics of an object ignoring the irrelevant details. The properties and behaviours of an object differentiate it from other objects of similar type and also help in classifying/grouping the objects.

Abstract class: is a restricted class that cannot be used to create objects (to access it, it must be inherited from another class).

Abstract method: can only be used in an abstract class, and it does not have a body. The body is provided by the subclass (inherited from).

EX: Consider a real-life example of a man driving a car. The man only knows that pressing the accelerators will increase the speed of the car or applying brakes will stop the car, but he does not know about how on pressing the accelerator the speed is increasing, he does not know about the inner mechanism of the car or the implementation of the accelerator, brakes, etc in the car. This is what abstraction is.

EX:

```
abstract class Vehicle {

    abstract void start(); // Abstract method to start the vehicle (no implementation)

}

class Car extends Vehicle {

    // Implementation of the abstract method start() from Vehicle

    @Override
```

```

void start() {

    System.out.println("Car is starting.");

}

// Main method to demonstrate abstraction

public static void main(String[] args) {

    Car myCar = new Car(); // Creating an instance of Car

    myCar.start(); // Output: Car is starting.

}

}

```

Interface

Interfaces are another method of implementing abstraction in Java. The key difference is that, by using interfaces, we can achieve 100% abstraction in Java classes. In Java or any other language, interfaces include both methods and variables but lack a method body. Apart from abstraction, interfaces can also be used to implement interfaces in Java

Q1. Why do we use abstract?

One key reason we use abstract concepts is to simplify complexity. Imagine trying to explain the entire universe with every single atom and star! Abstracts let us zoom out, grab the main ideas like gravity and energy, and make sense of it all without getting lost in the details.

Q2. What is the Difference between Encapsulation and Data Abstraction?

Here are some key difference b/w encapsulation and abstraction:

Encapsulation	Abstraction
Encapsulation is data hiding (information hiding)	Abstraction is detailed hiding (implementation hiding).
Encapsulation groups together data and methods that act upon the data	Data Abstraction deal with exposing the interface to the user and hiding the details of implementation
Encapsulated classes are Java classes that follow data hiding and abstraction	Implementation of abstraction is done using abstract classes and interface
Encapsulation is a procedure that takes place at the implementation level	abstraction is a design-level process
* Implemented using access modifiers (public, private, protected) to control access to class members (attributes and methods).	* Implemented using abstract classes (with abstract methods) and interfaces to define higher-level concepts and behaviors.

Q4. What is the Difference between Abstract Classes and Interfaces in Java?

Here are some key difference b/w Abstract Classes and Interfaces in Java:

Abstract Class	Interfaces
Abstract classes support abstract and Non-abstract methods	Interface supports have abstract methods only.
Doesn't support Multiple Inheritance	Supports Multiple Inheritance
Abstract classes can be extended by Java classes and multiple interfaces	The interface can be extended by Java interface only.
Abstract class members in Java can be private, protected, etc.	Interfaces are public by default.
Example: - public abstract class Vehicle{ public abstract void drive() }	Example: - public interface Animal{ void speak(); }

Advantages of Abstraction

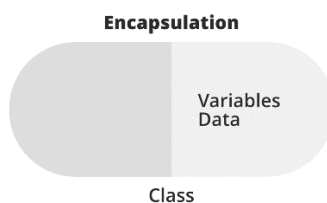
Here are some advantages of abstraction:

1. It reduces the complexity of viewing things.
2. Avoids code duplication and increases reusability.
3. It improves the maintainability of the application.
4. It improves the modularity of the application.
5. Improves code reusability and maintainability.
6. Hides implementation details and exposes only relevant information.
7. Provides a clear and simple interface to the user.
8. Increases security by preventing access to internal class details.

Encapsulation

Encapsulation is a way to restrict the direct access to some components of an object, so users cannot access state values for all of the variables of a particular object. Encapsulation can be used to hide both data members and data functions or methods associated with an instantiated class or object.

As in encapsulation, the data in a class is hidden from other classes, so it is also known as **data-hiding**.



EX:

```
public class Car {  
  
    private String make;  
  
    private String model;  
  
    private int year;
```

// Constructor to initialize the attributes

```
public Car(String make, String model, int year) {  
    this.make = make;  
    this.model = model;  
    this.year = year;}  
  
public String getMake() // Getter method to access the private 'make' attribute{  
    return make;  
}  
  
public String getModel() {  
    return model;  
}  
  
public int getYear() {  
    return year;  
}  
}
```

Advantages of Encapsulation

- **Data Hiding:** it is a way of restricting the access of our data members by hiding the implementation details. Encapsulation also provides a way for data hiding. The user will have no idea about the inner implementation of the class. It will not be visible to the user how the class is storing values in the variables.
- **Increased Flexibility:** Encapsulation improves the flexibility.
- **Reusability:** Encapsulation also improves the re-usability and is easy to change with new requirements.
- **Testing code is easy:** Encapsulated code is easy to test for unit testing.
- **Freedom to programmer in implementing the details of the system:** This is one of the major advantage of encapsulation that it gives the programmer freedom in implementing the details of a system.

Inheritance

Inheritance is an important pillar of OOP(Object-Oriented Programming). It is the mechanism in Java by which one class is allowed to inherit the features(fields and methods) of another class. In Java, Inheritance means creating new classes based on existing ones. A class that inherits from another class can reuse the methods and fields of that class. In addition, you can add new fields and methods to your current class as well.

EX:

```
class Vehicle {  
  
    void start() {  
  
        System.out.println("Vehicle is starting.");  
  
    }  
}  
  
// Child class Car inherits from Vehicle  
  
class Car extends Vehicle {  
  
    void accelerate() {  
  
        System.out.println("Car is accelerating.");  
  
    }  
  
    public static void main(String[] args) {  
  
        Car myCar = new Car();  
  
        myCar.start();    // Output: Vehicle is starting.  
  
        myCar.accelerate(); // Output: Car is accelerating.  
  
    }  
}
```

Why Do We Need Java Inheritance?

Code Reusability: The code written in the Superclass is common to all subclasses. Child classes can directly use the parent class code.

Method Overriding: Method Overriding is achievable only through Inheritance. It is one of the ways by which Java achieves Run Time Polymorphism.

Abstraction: The concept of abstract where we do not have to provide all details is achieved through inheritance. Abstraction only shows the functionality to the user.

Super Class/Parent Class: The class whose features are inherited is known as a superclass(or a base class or a parent class).

Sub Class/Child Class: The class that inherits the other class is known as a subclass(or a derived class, extended class, or child class).

Reusability: Inheritance supports the concept of “reusability”, i.e. when we want to create a new class and there is already a class that includes some of the code that we want, we can derive our new class from the existing class. By doing this, we are reusing the fields and methods of the existing class.

Advantages Of Inheritance in Java:

1. **Code Reusability:** Inheritance allows for code reuse and reduces the amount of code that needs to be written. The subclass can reuse the properties and methods of the superclass, reducing duplication of code.
2. **Abstraction:** Inheritance allows for the creation of abstract classes that define a common interface for a group of related classes. This promotes abstraction and encapsulation, making the code easier to maintain and extend.
3. **Class Hierarchy:** Inheritance allows for the creation of a class hierarchy, which can be used to model real-world objects and their relationships.
4. **Polymorphism:** Inheritance allows for polymorphism, which is the ability of an object to take on multiple forms. Subclasses can override the methods of the superclass, which allows them to change their behavior in different ways.

Polymorphism

Polymorphism is one of the core concepts of object-oriented programming (OOP) that describes situations in which something occurs in several different forms. In computer science, polymorphism describes the concept that you can access objects of different types through the same interface. Each type can provide its own independent implementation of this interface. In other words, polymorphism allows you to define one interface and have multiple implementations. The word “poly” means many and “morphs” means forms, So it means many forms.

Types of Java Polymorphism

In Java Polymorphism is mainly divided into two types:

- Compile-time Polymorphism
- Runtime Polymorphism

Explain the different types of polymorphism including the differences between them with example:

1. Compile-Time Polymorphism (Method Overloading):

- Defined at compile time by the compiler.
- Occurs when you have multiple methods in the same class with the same name but different parameter lists (number, type, or order of parameters).

EX:

```
class Calculator {  
  
    public int add(int a, int b) {  
  
        return a + b;  
  
    }  
  
    public double add(double a, double b) {  
  
        return a + b;  
  
    }  
}
```

Here, the `add` method is overloaded for `int` and `double` arguments. The compiler chooses the appropriate method based on the argument types used in the call.

2. Runtime Polymorphism (Method Overriding):

- Defined at runtime by the JVM (Java Virtual Machine).
- Runtime Polymorphism in Java is the process by which a call to an overridden method is resolved at runtime rather than at compile-time
This concept is also termed as dynamic method dispatch, an essential process in object-oriented programming.

EX:

```
class Animal {  
  
    public void makeSound() {  
  
        System.out.println("Generic animal sound");  
  
    }  
  
}  
  
class Dog extends Animal {  
  
    @Override // Override annotation for clarity  
  
    public void makeSound() {  
  
        System.out.println("Woof!");  
  
    }  
  
}
```

Here, `Dog` overrides the `makeSound` method from `Animal`. When you call `makeSound` on a `Dog` object referenced by an `Animal` variable, the JVM calls the `Dog` version at runtime due to the actual object's type.

Advantages of Polymorphism in Java

1. Increases code reusability by allowing objects of different classes to be treated as objects of a common class.
2. Improves readability and maintainability of code by reducing the amount of code that needs to be written and maintained.
3. Supports dynamic binding, enabling the correct method to be called at runtime, based on the actual class of the object.
4. Enables objects to be treated as a single type, making it easier to write generic code that can handle objects of different types.

Dynamic binding

Dynamic binding is also referred to as a run-time polymorphism(overriding). In overriding both parent and child classes have the same method. Dynamic binding is also called **Late binding**. In this type of binding, the functionality of the method call is not decided at compile-time. In other words, it is not possible to decide which piece of code will be executed as a result of a method call at compile-time. Overriding a method is a perfect example of dynamic binding in java.

Message passing

Message Passing in terms of computers is communication between processes. It is a form of communication used in object-oriented programming as well as parallel programming .Message passing in Java is like sending an object i.e. message from one thread to another thread. It is used when threads do not have shared memory and are unable to share monitors or any other shared variables to communicate.

Explanation of "open for extension, closed for modification" in the context of OOP with example:

Open for Extension: You can **add new features (behaviors)** to existing classes without modifying their core code. Think of adding new tools to a toolbox.E

Closed for Modification: You shouldn't directly **change the existing code** to introduce new behaviors. This keeps the code stable and avoids bugs.

Benefits:

- Easier to add new features without breaking old code.
- More maintainable and reusable code.

EX:

// Interface for operaton (Open for Extension)

```
interface Operation {  
    int perform(int a, int b);  
}
```

// Class (Closed for Modifcaton)

```
class Calculator {  
    public int calculate(int a, int b, Operation op) {  
        return op.perform(a, b);  
    }  
}
```

// Concrete operaton (Extension)

```
class Add implements Operation {  
    @Override  
    public int perform(int a, int b) {  
        return a + b;  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {
```

```

Calculator calc = new Calculator();

int sum = calc.calculate(5, 3, new Add()); // Use Add operaton

System.out.println(sum);
}

}

```

This code demonstrates:

- `Operation` interface defines behavior (Open for Extension).
- `Calculator` uses an `Operation` object but doesn't modify its behavior (Closed for Modification).
- `Add` implements `Operation` for addition (Extension).

You can add new operations (Subtract, Multiply) by creating new classes implementing `Operation`. This showcases OCP principles in a concise way.

Exceptions in Java.

What are Java Exceptions?

In Java, Exception is an unwanted or unexpected event, which occurs during the execution of a program, i.e. at run time, that disrupts the normal flow of the program's instructions. Exceptions can be caught and handled by the program. When an exception occurs within a method, it creates an object. This object is called the exception object. It contains information about the exception, such as the name and description of the exception and the state of the program when the exception occurred.

Describe the concept exception handling OOP with examples.

Exception Handling in Java

Exception Handling in Java is one of the effective means to handle runtime errors so that the regular flow of the application can be preserved. Exception handling in object-oriented programming (OOP) is a mechanism that allows developers to manage and respond to unexpected or exceptional situations that may occur during program execution. Exceptions are objects representing errors or abnormal conditions that disrupt the normal flow of the program. Let's explore the concept of exception handling in OOP with examples.

Key Concepts of Exception Handling in OOP:

Throwing Exceptions:

- Use `throw` to indicate an error condition within a method.

```
throw new IllegalArgumentException("Invalid input");
```

Catching Exceptions:

- Use `try-catch` blocks to handle exceptions gracefully.

```
try {  
    // Code that may throw an exception  
} catch (ExceptionType e) {  
    // Handle the exception  
}
```

Ex:

```
public class Main {  
    public static void main(String[] args) {  
        try {  
            int[] myNumbers = {1, 2, 3};  
            System.out.println(myNumbers[10]);  
        } catch (Exception e) {  
            System.out.println("Something went wrong.");  
        }  
    }  
}
```

Major reasons why an exception Occurs

- Invalid user input
- Device failure
- Loss of network connection
- Physical limitations (out-of-disk memory)
- Code errors
- Opening an unavailable file

Differentiate between error and exception:

- **Error:** An Error indicates a serious problem that a reasonable application should not try to catch.
- **Exception:** Exception indicates conditions that a reasonable application might try to catch.

Here's a quick explanation of the differences between errors and exceptions in Java:

Feature	Error	Exception
Recoverable	No, typically unrecoverable	Yes, can often be handled gracefully within the program
Cause	System or environment issues (out of memory)	Issues within the program logic (null pointer)
Subclasses	All are unchecked exceptions (not required to be caught at compile time)	Can be checked (must be caught or declared) or unchecked
Example	OutOfMemoryError, ThreadDeath	NullPointerException, ArrayIndexOutOfBoundsException

Describe the rule of exception handling in file I/O with examples:

Rule: When performing file I/O operations in Java, it's essential to handle potential exceptions using try-catch blocks. Exceptions occur due to various reasons such as runtime errors, invalid input, resource issues, logical errors, concurrency issues, and external dependencies.

EX:

```
public class FileReadExample {  
  
    public static void main(String[] args) {  
  
        try {  
  
            // Scanner for reading from a file  
  
            Scanner scanner = new Scanner(new File("data.txt"));  
  
            // Read and process data from the file (lines omitted for brevity)
```

```

        scanner.close(); // Close the scanner after use
    }

    catch (FileNotFoundException e) {

        System.out.println("Error: File 'data.txt' not found!"); }

    }

}

```

Explanation:

1. The `try` block attempts to open the file "data.txt" using a `Scanner`.
2. If successful, the code inside the `try` block would read and process data from the file (omitted for brevity).
3. The `catch` block handles the `FileNotFoundException` that might occur if the file doesn't exist. It prints an error message to `System.err`.

Benefits of Exception Handling:

- **Prevents program crashes:** By handling exceptions, you gracefully recover from errors and keep the program running.
- **Provides informative messages:** Exception messages pinpoint the issue, aiding debugging and user feedback.
- **Improves code maintainability:** Well-structured exception handling makes code more readable and easier to understand.
- **Security:** Enhances security by preventing information leakage and handling security-related exceptions.
- **Testing:** Supports effective testing by simulating and handling various error scenarios during testing phases.
- **Enhanced User Experience:** Improves user experience by handling errors gracefully and providing meaningful feedback.

Types of Exceptions

1. Built-in Exceptions

Built-in exceptions are the exceptions that are available in Java libraries. These exceptions are suitable to explain certain error situations. It is two types:

Checked Exceptions: Checked exceptions are called compile-time exceptions because these exceptions are checked at compile-time by the compiler.

Unchecked Exceptions: The unchecked exceptions are just opposite to the checked exceptions. The compiler will not check these exceptions at compile time.

2. User-Defined Exceptions:

Sometimes, the built-in exceptions in Java are not able to describe a certain situation. In such cases, users can also create exceptions, which are called ‘user-defined Exceptions’.

Methods to print the Exception information:

1. printStackTrace()

This method prints exception information in the format of the Name of the exception: description of the exception, stack trace.

Example:

```
class GFG {  
    public static void main (String[] args) {  
        int a=5;  
        int b=0;  
        try{  
            System.out.println(a/b);  
        }  
        catch (ArithmeticException e) {  
            e.printStackTrace();  
        }  
    }  
}
```

```
} }
```

Output

```
java.lang.ArithmeticException: / by zero  
at GFG.main(File.java:10)
```

2. toString()

The toString() method prints exception information in the format of the Name of the exception: description of the exception.

Example:

```
class GFG1 {  
    public static void main (String[] args) {  
        int a=5;  
        int b=0;  
        try{  
            System.out.println(a/b);  
        }  
        catch(ArithmeticException e){  
            System.out.println(e.toString());  
        }  
    }  
}
```

Output

```
java.lang.ArithmeticException: / by zero
```

3. getMessage()

The getMessage() method prints only the description of the exception.

Example:

```
class GFG1 {  
    public static void main (String[] args) {  
        int a=5;  
        int b=0;
```

```

        try{
            System.out.println(a/b);
        }
        catch(ArithmeticException e){
            System.out.println(e.getMessage());
        }
    }
}

```

Output

/ by zero

Java Threads

Threads allows a program to operate more efficiently by doing multiple things at the same time.

Threads can be used to perform complicated tasks in the background without interrupting the main program.

Why Threads are used?

Now, we can understand why threads are being used as they had the advantage of being lightweight and can provide communication between multiple threads at a Low Cost contributing to effective multi-tasking within a shared memory environment.

Multiple Threads

The main advantage of multiple threads is efficiency (allowing multiple things at the same time). For example, in MS Word. one thread automatically formats the document while another thread is taking user input. Another advantage is quick response, if we use multiple threads in a process and if a thread gets stuck due to lack of resources or an exception, the other threads can continue to execution, allowing the process (which represents an application) to continue to be responsive.

Multithreading in Java

Multithreading is a Java feature that allows concurrent execution of two or more parts of a program for maximum utilization of CPU. Each part of such program is called a thread. So, threads are light-weight processes within a process.

The Concept of Multitasking

Multitasking is the concurrent execution of multiple tasks (also known as processes) over a certain period of time. In the context of computer science, it refers to the ability of an operating system or a processor to execute multiple tasks or processes simultaneously.

1. **Process-Based Multitasking**
2. **Thread-Based Multitasking**

1. Process-Based Multitasking (Multiprocessing)

In this type of Multitasking, processes are heavyweight and each process was allocated by a separate memory area. And as the process is heavyweight the cost of communication between processes is high and it takes a long time for switching between processes as it involves actions such as loading, saving in registers, updating maps, lists, etc.

2. Thread-Based Multitasking

As we discussed above Threads are provided with lightweight nature and share the same address space, and the cost of communication between threads is also low.

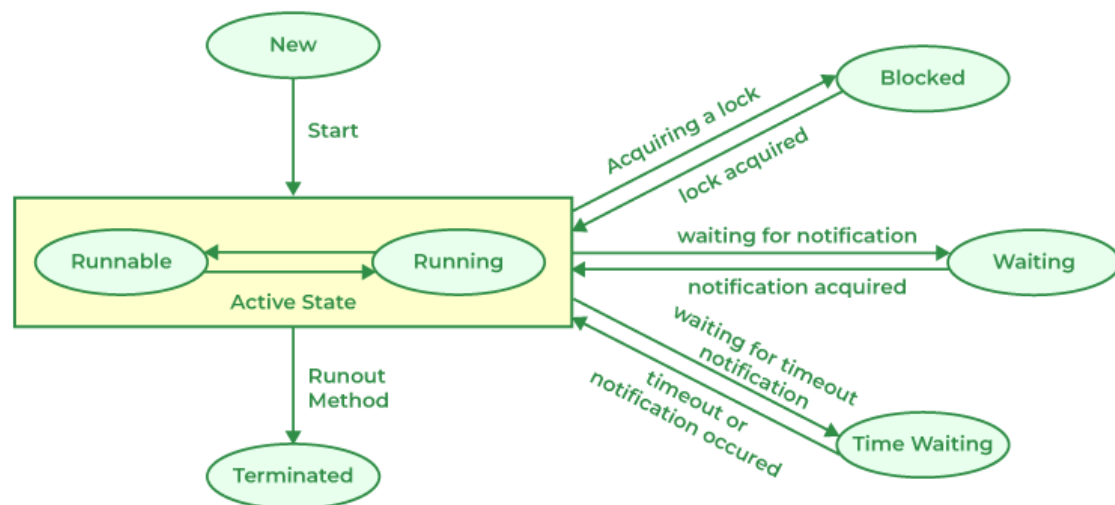
Lifecycle and States of a Thread in Java

A thread in Java at any point of time exists in any one of the following states. A thread lies only in one of the shown states at any instant:

1. New State
2. Runnable State

3. Blocked State
4. Waiting State
5. Timed Waiting State
6. Terminated State

The diagram shown below represents various states of a thread at any instant in time.



Life Cycle of a Thread

There are multiple states of the thread in a lifecycle as mentioned below:

1. **New Thread:** When a new thread is created, it is in the new state
2. **Runnable State:** A thread that is ready to run is moved to a runnable state.
3. **Blocked:** The thread will be in blocked state when it is trying to acquire a lock but currently the lock is acquired by the other thread.
4. **Waiting state:** The thread will be in waiting state when it calls wait() method or join() method.
5. **Timed Waiting:** A thread lies in a timed waiting state when it calls a method with a time-out parameter
6. **Terminated State:** A thread terminates because of either of the following reasons:
 - a. Because it exits normally. This happens when the code of the thread has been entirely executed by the program.

- b. Because there occurred some unusual erroneous event, like a segmentation fault or an unhandled exception.

Methods of Multithreading in Java

start()	The start method initiates the execution of a thread
currentThread()	The currentThread method returns the reference to the currently executing thread object.
run()	The run method triggers an action for the thread
isAlive()	The isAlive method is invoked to verify if the thread is alive or dead
sleep()	The sleep method is used to suspend the thread temporarily
yield()	The yield method is used to send the currently executing threads to standby mode and runs different sets of threads on higher priority
suspend()	The suspend method is used to instantly suspend the thread execution
resume()	The resume method is used to resume the execution of a suspended thread only
interrupt()	The interrupt method triggers an interruption to the currently executing thread class
destroy()	The destroy method is invoked to destroy the execution of a group of threads
stop()	The stop method is used to stop the execution of a thread

Note: In Java, a package is a namespace that organizes a set of related classes and interfaces

What Are Access Modifiers(**Access specifiers**) in Java?

Access modifiers in Java are keywords that determine the scope and visibility of classes , methods , variables , and constructors within an application . Java provides four main types of access modifiers : `public`, `private`, `protected`, and the default access (no modifier). The `public` modifier allows elements to be accessible from any other class in the application, regardless of the package. `Private` restricts access to the elements only within the class they are declared. `Protected` access modifier is accessible within package and outside the package but through inheritance only. Lastly, the default access (no modifier) limits the visibility to classes within the same package.

What are access modifiers in Java? Write down the purpose of it .

Access modifiers in Java define the visibility and accessibility of classes, methods, and fields. Here are the main access modifiers and their purposes:

1. **public**

- **Purpose:** Accessible from any class.
- **Example:**

```
public class MyClass {  
  
    public int publicVar = 10;  
  
    public void publicMethod() {  
  
        System.out.println("Public Method");  
  
    }  
}
```

2. **protected**

- **Purpose:** Accessible within the same package and by subclasses.
- **Example:**

```
public class MyClass {  
  
    protected int protectedVar = 20;  
  
    protected void protectedMethod() {  
  
        System.out.println("Protected Method");  
  
    }  
}
```

3. **default** (no modifier)

- **Purpose:** Accessible only within the same package.
- **Example:**

```
class MyClass {  
  
    int defaultVar = 40;  
  
    void defaultMethod() {  
  
        System.out.println("Default Method");  
  
    }  
}
```

4. **private**

- **Purpose:** Accessible only within the same class.
- **Example:**

```
public class MyClass {  
  
    private int privateVar = 30;  
  
    private void privateMethod() {  
  
        System.out.println("Private Method");  
  
    }  
}
```

How can we prevent a class from being inherited by other classes?

We can prevent a class from being subclassed by using the final keyword in the class's declaration. The final keyword is a non-access modifier used for classes, attributes and methods, which makes them non-changeable (impossible to inherit or override). Similarly, we can prevent a method from being overridden by subclasses by declaring it as a final method. An abstract class can only be subclassed; it cannot be instantiated.

Ex:

```
public final class MyClass {  
  
    // class members and methods  
  
}
```

Differences between Procedural and Object Oriented Programming

Procedural Programming

Procedural Programming can be defined as a programming model which is derived from structured programming, based upon the concept of calling procedure. Procedures, also known as routines, subroutines or functions, simply consist of a series of computational steps to be carried out. During a program's execution, any given procedure might be called at any point, including by other procedures or itself.

Languages used in Procedural Programming:

- FORTRAN, ALGOL, COBOL,
- BASIC, Pascal and C.

Object-Oriented Programming

Object-oriented programming can be defined as a programming model which is based upon the concept of objects. Objects contain data in the form of attributes and code in the form of methods. In object-oriented programming, computer programs are designed using the concept of objects that interact with the real world. Object-oriented programming languages are various but the most popular ones are class-based, meaning that objects are instances of classes, which also determine their types.

Languages used in Object-Oriented Programming:

- Java, C++, C#, Python,
- PHP, JavaScript, Ruby, Perl,
- Objective-C, Dart, Swift, Scala.

Procedural Programming vs Object-Oriented Programming

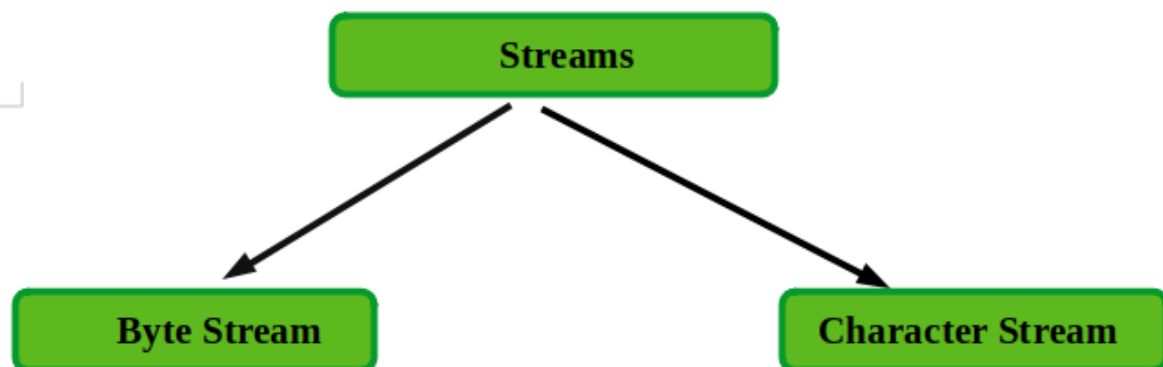
Below are some of the differences between procedural and object-oriented programming:

Procedural Oriented Programming	Object-Oriented Programming
In procedural programming, the program is divided into small parts called functions .	In object-oriented programming, the program is divided into small parts called objects .
Procedural programming follows a top-down approach .	Object-oriented programming follows a bottom-up approach .
There is no access specifier in procedural programming.	Object-oriented programming has access specifiers like private, public, protected, etc.
Adding new data and functions is not easy.	Adding new data and function is easy.
In procedural programming, overloading is not possible.	Overloading is possible in object-oriented programming.
In procedural programming, there is no concept of data hiding and inheritance.	In object-oriented programming, the concept of data hiding and inheritance is used.
In procedural programming, the function is more important than the data.	In object-oriented programming, data is more important than function.
Procedural programming is based on the unreal world .	Object-oriented programming is based on the real world .
Procedural programming is used for designing medium-sized programs.	Object-oriented programming is used for designing large and complex programs.
Code reusability absent in procedural programming,	Code reusability present in object-oriented programming.
Examples: C, FORTRAN, Pascal, Basic, etc.	Examples: C++, Java, Python, C#, etc

Difference Between InputStream and OutputStream in Java

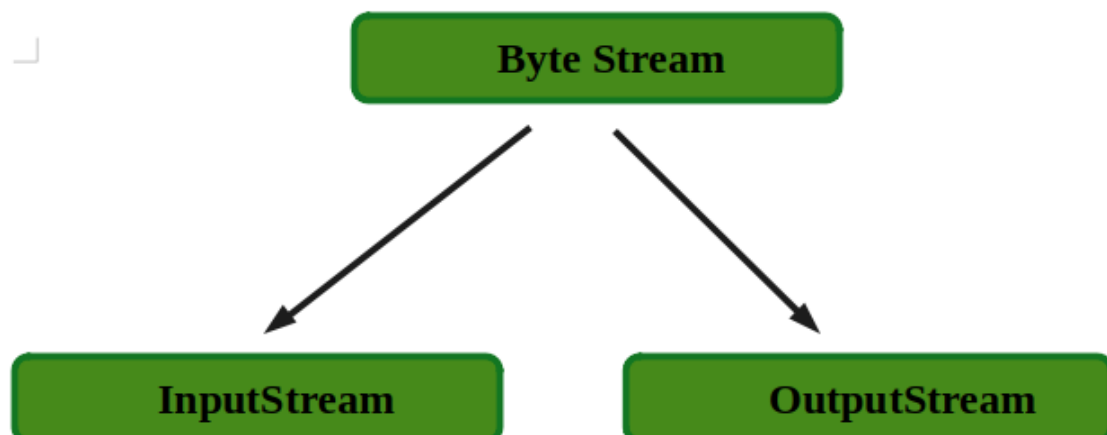
In Java, the InputStream and OutputStream classes, as well as the Reader and Writer classes, are part of the I/O (Input/Output) framework.

A **stream** can be defined as the sequence of data or continuous flow of data. Streams are a clear way to deal with Input/Output. Streams are of two types as Depicted below:



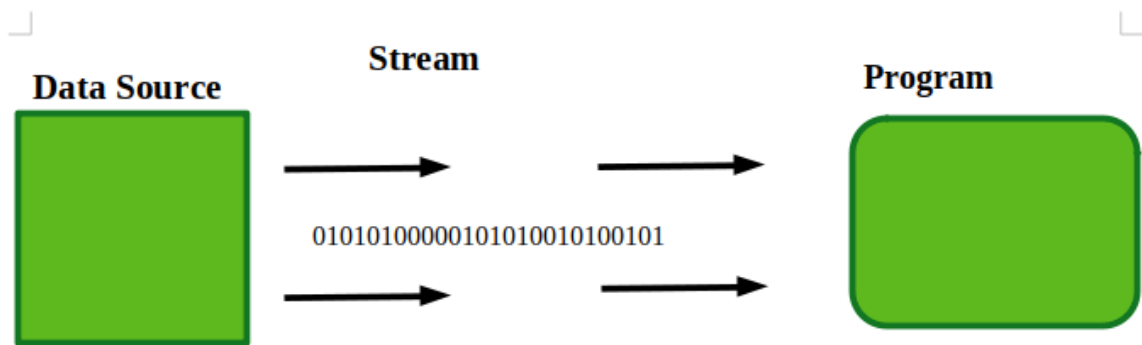
In the above diagram, our InputStream and OutputStream will reside in Byte Stream. So let's discuss byte Stream.

1. Byte Stream: Byte Stream provides a convenient way of handling the input and output of byte. The byte stream is further divided into various classes but the top hierarchy Classes are depicted below:

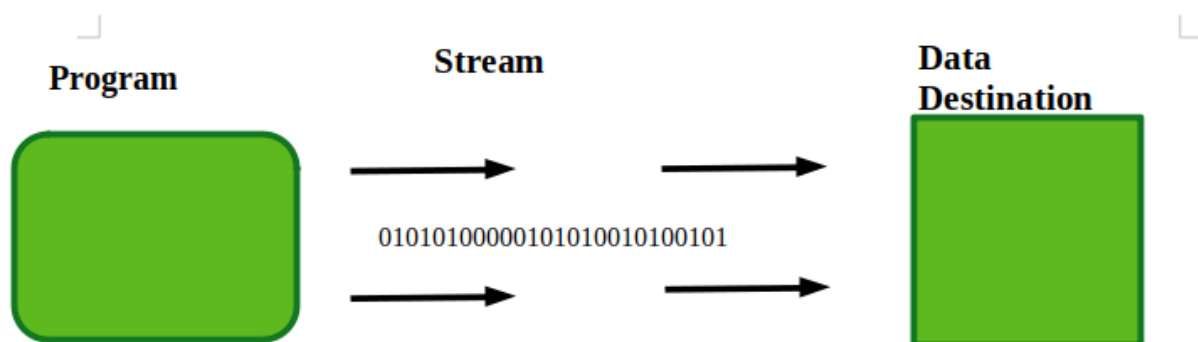


1.1 InputStream: InputStream is an abstract class of Byte Stream that describe stream input and it is used for reading and it could be a file, image, audio, video,

webpage, etc. it doesn't matter. Thus, `InputStream` read data from source one item at a time.



1.2 OutputStream: `OutputStream` is an abstract class of Byte Stream that describes stream output and it is used for writing data to a file, image, audio, etc. Thus, `OutputStream` writes data to the destination one at a time.



Difference between `InputStream` and `OutputStream`

InputStream	OutputStream
1. It is an abstract class that describes Stream Input.	1. It is an abstract class that describes Stream Output.
2. <code>InputStream</code> Read data from the source once at a time.	2. <code>OutputStream</code> Write Data to the destination once at a time.
3. <code>InputStream</code> consist of method which performs: Read next byte of data from the input stream and return - 1 at the end of the file:	3. Output Stream consists of methods which perform: Write a byte to current Outputstream : public void write(int) throws IOException

public abstract int read()throws IOException • Close current InputStream: public int available()throws IOException • Returns an estimate of the number of bytes that can be read from the current input stream: public void close()throws IOException	• Write array of byte to current output stream : public void write(byte[])throws IOException • Flushes the current OutputStream: public void flush()throws IOException • Close current Output Stream. : public void close()throws IOException
4. Types of InputStream are: • FileInputStream • ByteArrayInputStream • FilterInputStream • ObjectInputStream In these types the most important and mostly used type is FileInputStream.	4. Types of OutputStream are: • FileOutputStream • ByteArrayOutputStream • FilterOutputStream • ObjectOutputStream In these types the most important and mostly used type is FileOutput Stream.

What are the differences between association, aggregation and composition?

Describe the association and composition concept with examples in OOP

Association, aggregation, and composition are concepts used in object-oriented programming (OOP) to represent relationships between classes or objects. Here are the key differences between them:

1. Association:

- **Definition:** Association represents a relationship between two or more objects where each object has its own lifecycle, and there is no ownership.
- **Nature:** It is a very general relationship that can be one-to-one, one-to-many, or many-to-many.
- **Example:** A teacher and a student are associated because a teacher teaches one or more students, but the teacher does not own the student nor vice versa.

2. Aggregation:

- **Definition:** Aggregation is a specialized form of association where objects are associated with each other, but there is a "whole-part" relationship, where the parts can exist independently of the whole.
- **Ownership:** The part (child) object can belong to multiple different parent objects or none.
- **Example:** A car and an engine have an aggregation relationship because an engine can exist independently (in other cars or as a spare part) and can be swapped between cars.

3. Composition:

- **Definition:** Composition is a stronger form of aggregation where the part (child) object cannot exist or is destroyed when the owner (parent) object is destroyed.
- **Ownership:** The part is a crucial part of the whole and cannot exist without the whole.
- **Example:** A House has Rooms. The Rooms are part of the House, and if the House is demolished, all its Rooms are also destroyed.

HashSet: HashSet is an implementation of the Set interface that uses a hash table for storage.

LinkedHashSet: Linkedhashset extends HashSet and maintains a doubly-linked list running through all of its entries.

Explain why you would choose a LinkedHashSet (or LinkedHashMap) over a HashSet (or HashMap, respectively).

Choosing a LinkedHashSet over a HashSet, or a LinkedHashMap over a HashMap, depends on the specific requirements of our use case. Here are some reasons why you might opt for the Linked variants:

- **Iteration Order:** LinkedHashSet and LinkedHashMap maintain insertion order, useful for when the sequence of elements matters.
- **Performance Trade-offs:** Slightly higher memory usage and overhead due to maintaining a doubly-linked list.
- **Access Order in LinkedHashMap:** Can maintain access order for LRU cache implementations.
- **Easier Debugging:** Predictable order can simplify debugging and logging.

Explain the Is-a Relationships and Has-a Relationships in OOP. Let's say a system contains the following objects. Can you identify the relationship among these objects?

1.Bike	2.Pulser	3.Engine
4.Building	5.Window	6.Door
7.Student	6.Teacher	8.Person

The prompt asks to identify the relationships between the objects in a system containing a bike, a pulsar, an engine, a building, a window, a door, a student, a teacher, and a person.

In object-oriented programming (OOP), there are two main types of relationships between objects: is-a relationships and has-a relationships.

- **Is-a relationship** is a hierarchical relationship between classes. In this type of relationship, one class inherits properties and methods from another class. For example, in the prompt, a student is-a person. This means that the student class inherits properties and methods from the person class.
- **Has-a relationship** is a compositional relationship between classes. In this type of relationship, one class has an instance of another class as a member variable. For example, a building has-a door. This means that the building class can have a member variable of type door.

Let's look at the relationships between the objects in the prompt:

- **Bike** - Does not have an is-a or has-a relationship with any of the other objects in the list.
- **Pulsar** - Does not have an is-a or has-a relationship with any of the other objects in the list. It likely refers to a brand of a motorcycle, not relevant to the other objects.
- **Engine** - An engine has-a relationship with a bike. A bike can have an engine as a part.
- **Building** - A building has-a relationship with a door and a window. A building can have doors and windows as parts.
- **Window** - Does not have an is-a or has-a relationship with any of the other objects in the list, other than the building it can be a part of.
- **Door** - Does not have an is-a or has-a relationship with any of the other objects in the list, other than the building it can be a part of.
- **Student** - Is-a person. A student inherits properties and methods from the person class.
- **Teacher** - Is-a person. A teacher inherits properties and methods from the person class.

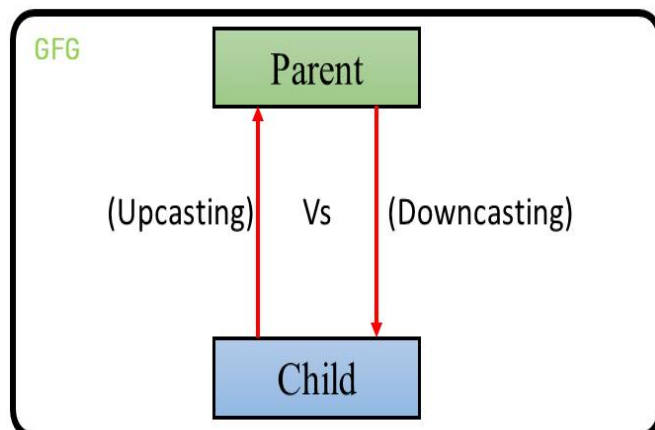
Explain up-casting and down casting with appropriate examples.

Up-casting

Upcasting is the typecasting of a child object to a parent object. Upcasting can be done implicitly.

Example

```
class Animal {  
    void makeSound() {  
        System.out.println("Animal sound");  
    }  
}  
  
class Dog extends Animal {  
    void makeSound() {  
        System.out.println("Bark");  
    }  
}
```



Typecasting is one of the most important concepts which basically deals with the conversion of one data type to another data type implicitly or explicitly

```

public class Main {

    public static void main(String[] args) {

        Dog dog = new Dog();

        Animal animal = dog; // Up-casting: Dog to Animal

        animal.makeSound(); // Output: Bark

    }

}

```

Explanation

- **Up-casting:** `Animal animal = dog;`
 - A `Dog` object is assigned to an `Animal` reference.
 - `animal.makeSound()` ; calls the `Dog` class's `makeSound` method, printing "Bark".

Down-casting

Similarly, downcasting means the typecasting of a parent object to a child object. Downcasting cannot be implicit.

Example

```

class Animal {

    void makeSound() {

        System.out.println("Animal sound");

    }

}

```

```

class Dog extends Animal {

    void makeSound() {

        System.out.println("Bark");

    }

    void fetch() {

        System.out.println("Fetching...");

    }

}

```

```

public class Main {

    public static void main(String[] args) {

        Animal animal = new Dog(); // Up-casting: Dog to Animal

        animal.makeSound();    // Output: Bark


        Dog dog = (Dog) animal; // Down-casting: Animal to Dog

        dog.fetch();    // Output: Fetching...

    }

}

```

Explanation

- **Up-casting:** `Animal animal = new Dog();`
 - A `Dog` object is assigned to an `Animal` reference.
 - `animal.makeSound();` calls the `Dog` class's `makeSound` method, printing "Bark".
- **Down-casting:** `Dog dog = (Dog) animal;`
 - Before down-casting, `instanceof` is used to check if `animal` is a `Dog`.
 - If true, `animal` is cast back to a `Dog` reference, allowing `dog.fetch();` to be called, printing "Fetching...".

These examples demonstrate the concepts of up-casting and down-casting in a simple and clear manner.

Example: Analyzing the system for online ticket booking, the following requirements were

identified -

1. There can be three types passenger: General, Premium and New
2. Passenger can register into the system with his -NID, Name, Email, Mobile Number,

and Password

3. Passenger can login into the system with his email and password
4. Passenger can buy
ticket
5. Ticket should be confirmed by the System Admin
6. Passenger can pay the ticket price online after being confirmed
7. Online payment should be using the “PAYBD” payment gateway
8. Passenger can cancel a booking
9. Passenger can request for return money for the canceled booking

a) Identify the classes with their attributes and methods.

b) Create the Class-based Model with proper connections.

Ans

a) Identify the classes with their attributes and methods

From the system analysis for online ticket booking, here are the classes identified with their attributes and methods:

- **Passenger**
 - Attributes:
 - NID (National Identification Number)
 - name
 - email
 - mobile number
 - password
 - ticket (object) - This can be linked to the Ticket class
 - Methods:
 - register() - Allows a passenger to register into the system with their details
 - login() - Enables a passenger to log in using their email and password
 - buyTicket() - Initiates the process of buying a ticket for the passenger

- cancelBooking() - Allows a passenger to cancel a booking they've made
- requestRefund() - Enables a passenger to request a refund for a cancelled booking
- **Ticket**
 - Attributes:
 - ID (unique identifier for the ticket)
 - price (cost of the ticket)
 - confirmationStatus (boolean - indicates whether the ticket is confirmed or not)
 - Methods:
 - confirmTicket() (This could be done in the Passenger class as well) - Marks the ticket as confirmed by the system admin
 - payTicketOnline() - Allows the passenger to pay for the ticket electronically (implementation details might reside elsewhere in the system)
- **System Admin**
 - Methods:
 - confirmTicket() - Confirms a ticket booking made by a passenger

b) Class-Based Model:

	Passenger

- nid: string	
- name: string	
- email: string	

```

| - mobile_number: string      |
| - password: string          |
| - bookings: list            |
|
|-----|
| + login(email: string, password: string) |
| + buy_ticket(ticket: Ticket)             |
| + cancel_booking(booking_id: string)     |
| + request_refund(booking_id: string)     |
|
|-----|
|
| ^
|
| uses 1
|
| V
|
|-----|
| Ticket                                |
|
|-----|
| - ticket_id: string                  |
| - passenger: Passenger                |
| - price: float                        |
| - status: string                      |
|
|-----|
| + confirm_ticket()                   |
| + make_payment()                     |
|
|-----|

```

```

      ^
      |
confirmed by 1
      |
      v
-----
|      Admin      |
-----
| - admin_id: string |
| - name: string    |
-----
| + confirm_ticket(ticket: Ticket) |
-----

```

Example: A program which decrypts files under the Swap Encryption Scheme by swapping pairs of characters is given below. Some code has been omitted and you need not to understand the operation of the algorithm.

```

1  class Swapper extends Reader {
2      private final PushbackReader pushBack;
3      Swapper(PushbackReader p) { pushBack = p; }
4
5      @Override
6      public int read(char[] cbuf, int off, int len) {
7          int r = wrap.read(cbuf, off, len);
8          if (r % 2 == 1) { pushBack.unread(cbuf, off + --r, 1); }
9          for (int i = 0; i < r; i += 2) { swap(cbuf, i, i + 1); }
10         return r;
11     }
12 }
13
14 class Decryptor {
15     static List<String> read(String fileName) {
16         try (BufferedReader r = new BufferedReader(new Swapper(
17             new PushbackReader(new FileReader(fileName)))) {
18             return readLines(r);
19         }
20     }
21 }

```

a) The four principles of object-oriented programming are encapsulation, abstraction, inheritance and polymorphism. Explain how the program makes use of each of them, with reference to specific lines in the code.

Ans

I'd be glad to explain how the provided code utilizes the four main principles of object-oriented programming (OOP):

Encapsulation (Lines 2-3):

- The `Swapper` class encapsulates the `PushbackReader` object within the private member `pushBack`. This restricts direct access to the reader, promoting data protection and preventing unintended modifications.

Abstraction (Lines 5-11):

- The `read(char[] ebuf, int off, int len)` method hides the underlying decryption logic (swapping characters) from the user. They only need to provide the buffer, offset, and length, simplifying interaction with the decryption process.

Inheritance (Line 1):

- The `Swapper` class inherits from the `Reader` class (though not explicitly shown). This allows `Swapper` to access and potentially override methods defined in `Reader`, providing a foundation for character-based input operations.

Polymorphism (Not directly evident):

- While not explicitly demonstrated in this code snippet, polymorphism could potentially be employed through inheritance if the `Swapper` class were further extended. Subclasses could inherit the `read` method and provide specialized decryption functionality for different encryption schemes beyond simple character swapping.

Additional Notes:

- The provided code focuses more on encapsulation and abstraction as the core functionalities.
- Inheritance is present, but its usage is implicit due to the missing parent class definition.
- Polymorphism might be a future consideration for handling different encryption variations.

Example:

a) Create three classes with the following names “Student”, “Teacher” and “Officer”. Write the attributes and methods of the classes to perform an examination. (Include the attributes and methods based on your real-life examination experience).

Attributes and Methods:

1. Student:

○ Attributes:

- `studentID`: A unique identifier for the student.
- `name`: The name of the student.
- `courses`: A list of courses the student is enrolled in.
- `marks`: A list of marks the student received.

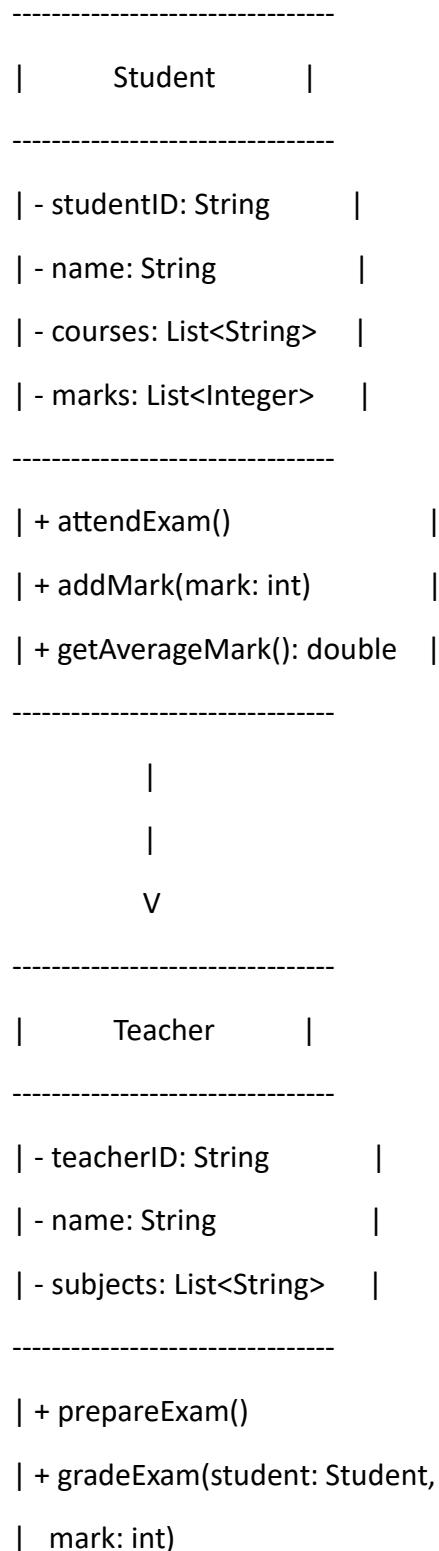
- **Methods:**
 - `attendExam()` : Simulates the student attending an exam.
 - `addMark(int mark)` : Adds a mark to the student's list of marks.
 - `getAverageMark()` : Calculates and returns the average mark of the student.
- 2. **Teacher:**
 - **Attributes:**
 - `teacherID`: A unique identifier for the teacher.
 - `name`: The name of the teacher.
 - `subjects`: A list of subjects the teacher teaches.
 - **Methods:**
 - `prepareExam()` : Simulates the teacher preparing the exam.
 - `gradeExam(Student student, int mark)`: Grades the exam for a given student and adds the mark to the student's record.
- 3. **Officer:**
 - **Attributes:**
 - `officerID`: A unique identifier for the officer.
 - `name`: The name of the officer.
 - **Methods:**
 - `organizeExam()` : Simulates the officer organizing the exam.
 - `collectPapers(Student student)`: Collects exam papers from students.
 - `publishResults(List<Student> students)`: Publishes the exam results for a list of students, displaying their average marks.

These classes and methods represent a simple model of the roles and interactions involved in conducting an examination in a real-life scenario.

b) Draw the class-based model from this(a) with proper relationship. Will there be any other classes necessary to complete the overall procedure? Explain your answer.

Class-Based Model Diagram

Below is a textual representation of the class-based model showing the relationships between the `Student`, `Teacher`, and `Officer` classes. Each class is defined with its attributes and methods, and the interactions are shown.



```

-----
|
|
V
-----
|      Officer      |
-----
| - officerID: String |
| - name: String     |
-----
| + organizeExam()    |
| + collectPapers(student: |
|   Student)          |
| + publishResults(students: |
|   List<Student>)      |
-----

```

Question: When do we use a protected access modifier?

Ans: In Java, use `protected` to grant access to:

Access within the same class: Members marked as protected can be accessed from within the class where they are declared.

Access within the same package: protected members can be accessed from any class within the same package.

Access from subclasses: protected members can be accessed from subclasses, even if they are in a different package.

No access from outside the class hierarchy: protected members cannot be accessed from outside the class hierarchy (i.e., from unrelated classes).

Think of it as a way to share functionality within a family (subclasses) and close neighborhood (same package) while keeping things private outside.

Interface

Interfaces are another method of implementing abstraction in Java. The key difference is that, by using interfaces, we can achieve 100% abstraction in Java classes. In Java or any other language, interfaces include both methods and variables but lack a method body. Apart from abstraction, interfaces can also be used to implement interfaces in Java

EX:

// Define the Animal interface

```
interface Animal {  
    void makeSound(); // Abstract method to make a sound  
}
```

// Implementing the Animal interface in Dog class

```
class Dog implements Animal {  
    // Implementing the makeSound method  
    @Override  
    public void makeSound() {  
        System.out.println("Woof!");  
    }  
}
```

// Main method to demonstrate interface implementation

```
public static void main(String[] args) {  
    Dog myDog = new Dog(); // Creating an instance of Dog  
    myDog.makeSound(); // Output: Woof!  
}  
}
```

Built-in Exceptions:

Built-in exceptions are the exceptions that are available in Java libraries. These exceptions are suitable to explain certain error situations. Below is the list of important built-in exceptions in Java.

1. **ArithmeticException:** It is thrown when an exceptional condition has occurred in an arithmetic operation.
2. **ArrayIndexOutOfBoundsException:** It is thrown to indicate that an array has been accessed with an illegal index. The index is either negative or greater than or equal to the size of the array.
3. **ClassNotFoundException:** This Exception is raised when we try to access a class whose definition is not found
4. **FileNotFoundException:** This Exception is raised when a file is not accessible or does not open.
5. **IOException:** It is thrown when an input-output operation failed or interrupted
6. **InterruptedException:** It is thrown when a thread is waiting, sleeping, or doing some processing, and it is interrupted.
7. **NoSuchFieldException:** It is thrown when a class does not contain the field (or variable) specified
8. **NoSuchMethodException:** It is thrown when accessing a method that is not found.
9. **NullPointerException:** This exception is raised when referring to the members of a null object. Null represents nothing
10. **NumberFormatException:** This exception is raised when a method could not convert a string into a numeric format.
11. **RuntimeException:** This represents an exception that occurs during runtime.
12. **StringIndexOutOfBoundsException:** It is thrown by String class methods to indicate that an index is either negative or greater than the size of the string
13. **IllegalArgumentException :** This exception will throw the error or error statement when the method receives an argument which is not accurately fit to the given relation or condition. It comes under the unchecked exception.
14. **IllegalStateException :** This exception will throw an error or error message when the method is not accessed for the particular operation in the application. It comes under the unchecked exception.

Describe association and composition in object-oriented programming (OOP) with examples:

Association: Association represents a relationship between two or more objects where each object has its own lifecycle, and there is no ownership.

- A general connection between two classes where one class "uses" another class.
- Represents a "has-a" relationship, but there's no ownership dependency.
- Objects in the association can exist independently of each other.
- Cardinalities (relationships between quantities) can be one-to-one, one-to-many, many-to-one, or many-to-many.

EX:

```
class Course {  
  
    private String name;  
  
    private int code;  
  
}  
  
class Student {  
  
    private String name;  
  
    private List<Course> courses = new ArrayList<>();  
  
    public void enroll(Course course) {  
  
        courses.add(course);  
  
    }  
  
}
```

In this example, the `Student` class is associated with the `Course` class. A `Student` object can enroll in multiple `Course` objects, and the `Course` objects can exist independently of the `Student` object.

Composition : Composition is a stronger form of aggregation where the part (child) object cannot exist or is destroyed when the owner (parent) object is destroyed.

- A stronger form of association where one class (the whole) "has-a" part (another class) that's dependent on its existence.
- Represents a "part-of" relationship, with a stricter ownership dependency.

- The contained class's lifetime is tied to the container class.
- The container class is responsible for creating and destroying the contained object(s)

EX:

```
class Door {

    public void open() { System.out.println("Door opened"); }

    public void close() { System.out.println("Door closed"); }

}

class Window {

    public void open() { System.out.println("Window opened"); }

    public void close() { System.out.println("Window closed"); }}

}

class House {

    private final Door door = new Door();

    private final Window window = new Window();

    public void open() {

        door.open();

        window.open();

    }

    public void close() {

        door.close();

        window.close();

    }

}
```

In this example, the `House` class is composed of `Door` and `Window` objects. The `House` object creates and owns the `Door` and `Window` objects, and the

lifecycle of the `Door` and `Window` objects is dependent on the lifecycle of the `House` object.

Java Inheritance Types

Single Inheritance: A subclass inherits from a single superclass.

Multilevel Inheritance: A subclass inherits from a superclass, which in turn inherits from another superclass.

Hierarchical Inheritance: Multiple subclasses inherit from a single superclass.

Multiple Inheritance (not supported in Java): A subclass inherits from multiple superclasses.

Hybrid Inheritance: A combination of two or more of the above inheritance types

